A Secure Multi-Tenant Identity, Authorization and Enterprise Access Platform

Week 1 Detailed Design and Security Foundation

Krishna Teja (CS23BTECH11028)
Jaya Prakash (CS23BTECH11015)

Indian Institute of Technology Hyderabad

September 2026

Abstract

This report presents the detailed design work for Week 1 of the Secure Multi-Tenant Identity, Authorization and Enterprise Access Platform project. The Week 0 report established the problem, studied related systems and identified seven major milestones. The mentor review agreed that the problem understanding was strong, but pointed out that the proposed implementation was too broad for a two member team working for eight weeks.

This report therefore focuses on making the project more practical and more precise. It defines the implementation boundary, compares alternative architectures, proposes a modular monolith architecture, defines the identity and tenant data model, describes tenant isolation, defines RBAC, ABAC and ReBAC usage, specifies a session policy, introduces a threat model and security invariants, and creates an initial requirements traceability and authorization plan. Agent identity is retained as an extension of the core authorization model instead of allowing it to take priority over the required M1 to M7 work.

The main idea is not to reduce the security goals of the project. Instead, the design reduces unnecessary infrastructure work so that the team can spend more time proving the security properties of the system.

Contents

1 Introduction **4**

2 What Has Changed From the Week 0 Report **4**

3 Scope Boundary **5**
3.1 What the Project Will Build 5
3.2 What Will Be Integrated 5
3.3 What Will Be Simulated 6
3.4 What Is Out of Scope 6

4 Architecture Alternatives **6**
4.1 Alternative 1: Eight Independent Microservices 6
4.2 Alternative 2: Traditional Monolith 7
4.3 Alternative 3: Modular Monolith 7
4.4 Comparison 7
4.5 Chosen Architecture 7

5 Authentication Architecture **8**
5.1 Why Not Build a Full OIDC Server 8
5.2 External Identity Provider Option 8
5.3 Responsibilities 8

6 Identity and Tenant Data Model **9**
6.1 User 9
6.2 AuthenticationIdentity 9
6.3 Profile 9
6.4 Organization 10
6.5 Membership 10
6.6 Role 10

7 Tenant Isolation **11**
7.1 The Problem 11
7.2 Trusted Tenant Context 11
7.3 Tenant Isolation Alternatives 11

8 Authorization Design **12**
8.1 RBAC 12
8.2 ABAC 12
8.3 ReBAC 12
8.4 Unified Authorization Interface 13

9 Session and Device Policy **13**
9.1 Access Tokens 13
9.2 Refresh Tokens 13
9.3 Replay Detection 14
9.4 Device Revocation 14
9.5 Account Reset 14

10 Threat Model and Security Invariants **14**
10.1 Assets 14
10.2 Threat Actors 15
10.3 Security Invariants 15

11 Agent Identity and Delegation **15**

12 Enterprise Provisioning **16**

13 Consent and Audit **16**

14 Initial Authorization Scenarios **17**

15 Requirements Traceability **18**

16 Week 1 Implementation Skeleton **19**

17 Measurable Success Criteria **20**

18 Questions From the Mentor and Design Answers **20**
18.1 Can one user belong to two organizations? 20
18.2 What prevents a tenant ID from being replaced by a client? 20
18.3 What happens to an access token after device revocation? 21
18.4 Which example needs ReBAC? 21
18.5 Which OIDC responsibilities are delegated? 21
18.6 How is organization provisioning idempotent? 21
18.7 Can an agent exceed its user's permissions? 21
18.8 What happens if authorization is unavailable? 21
18.9 What is the smallest end to end demonstration? 21

19 Plan for the Remaining Weeks **21**

20 Conclusion **22**

1 Introduction

The Week 0 report defined the project as a standalone identity and access platform for a multi-tenant learning SaaS. The main requirement was to separate authentication, user profile management, tenant membership and authorization. The project also includes enterprise access, session management, consent, auditing and security testing.

The original report already identified the main project milestones. M1 covers the identity and tenant model. M2 covers authentication and federation. M3 covers sessions and devices. M4 covers fine-grained authorization. M5 covers enterprise provisioning. M6 covers consent and auditability. M7 covers security validation.

The report also included AI agents as first class principals after the mentor suggested this direction. The agent idea is kept in this design, but it will be implemented after the core identity and authorization system is stable.

The main issue found during the Week 0 review was not the problem statement itself. The issue was the size of the proposed implementation. The report described eight logical components, and if each of them became an independently deployed service, a large amount of project time would be spent on service communication, deployment and distributed state. The mentor therefore recommended that these components should initially be treated as logical modules inside a modular monolith, with an external identity provider or policy component used where it makes sense.

This Week 1 report converts the original proposal into a more detailed engineering plan.

2 What Has Changed From the Week 0 Report

The Week 0 report was mainly an understanding and related work document. It answered questions such as why identity should be separated from a learning application, what existing systems are available, and which standards are relevant.

The Week 1 work has a different purpose. It answers implementation questions.

Area	Week 0 approach	Week 1 decision
Architecture	Eight logical components were proposed.	Keep the eight logical components, but implement them first as modules in one backend.
Authentication	Authentication service was described as an OIDC capable service that issues tokens.	Use an existing identity provider for standards heavy OIDC work and implement the project specific identity and access logic ourselves.
Data model	Entities were listed.	Define relationships, keys, cardinalities and security rules.
Tenant isolation	Identified as a major goal.	Define exactly how tenant context is derived and checked.

Authorization	RBAC, ABAC and ReBAC were proposed together.	Define a reason for each model and combine them through one authorization interface.
Sessions	Token expiry, rotation and revocation were mentioned.	Define measurable lifetimes and exact revocation and replay behaviour.
SCIM	A fairly broad SCIM interface was proposed.	Start with a minimum enterprise provisioning slice and treat full SCIM as an extension.
Security testing	Most testing was placed near the final phase.	Add security tests to every milestone from the beginning.
Traceability	References were mapped to milestones.	Requirements will be mapped to components, tests and evidence.

The changes are therefore mostly changes in precision and scope. The core project problem remains the same.

3 Scope Boundary

3.1 What the Project Will Build

The project will implement the following project specific functionality:

- User and application profile management.
- Organization and tenant management.
- Organization memberships and tenant scoped roles.
- Session and device records.
- Authorization decisions using RBAC, ABAC and selected ReBAC rules.
- Consent records and consent changes.
- Structured audit events.
- Enterprise organization provisioning.
- Agent principals and user to agent delegation.
- Security tests for tenant isolation, authorization, token replay and malformed claims.

3.2 What Will Be Integrated

The project should not attempt to recreate every part of a mature identity provider.

An existing identity provider such as Keycloak can be used for the standards heavy authentication part of the demonstration. The project will integrate with it through OIDC. This allows the team to demonstrate a realistic authentication flow without making the implementation of a secure authorization server the whole project.

The project will still own the application level user mapping, tenant membership, authorization, session and device policy, consent, audit and agent delegation.

3.3 What Will Be Simulated

Some enterprise functions can be simulated because real enterprise credentials are not required by the project.

The following can be simulated:

- An enterprise identity provider.
- Enterprise SSO configuration.
- An enterprise employee joining the learning SaaS.
- An employee being deactivated.
- A downstream learning application.
- Mobile and web client behaviour where a full production application is not needed.

The Week 0 assumptions already allow external identity providers to be mocked when real credentials are not available, as long as the security semantics remain realistic.

3.4 What Is Out of Scope

The following will not be treated as core requirements:

- Building the learning content service.
- Payment systems.
- Video streaming.
- Course grading logic itself.
- A production scale distributed microservice deployment.
- A complete replacement for Keycloak, Okta or another enterprise identity provider.

This boundary is important because the goal is to demonstrate identity and access engineering, not to build an entire SaaS product.

4 Architecture Alternatives

4.1 Alternative 1: Eight Independent Microservices

The first option is to deploy each logical component as its own service.

```
Authentication Service
User/Profile Service
Tenant Service
Session Service
Authorization Service
SCIM Service
Consent/Audit Service
Agent Service
```

This architecture is close to the original component description. It gives strong deployment separation and would allow each service to scale independently.

However, it also introduces a lot of work that is not directly required. Services would need to communicate securely, service failures would need to be handled, distributed transactions would become a concern, and local development would be harder.

For a two member team, this creates a real risk that the final system contains many services but the actual security demonstrations remain incomplete.

4.2 Alternative 2: Traditional Monolith

A traditional monolith puts all functionality in one backend without strong module boundaries.

This is easy to develop, but it creates a different risk. Authentication, tenant checks and authorization can become mixed together. A developer could easily start putting tenant checks inside arbitrary business code. This would make the separation that the project is supposed to demonstrate less clear.

4.3 Alternative 3: Modular Monolith

A modular monolith keeps one deployable backend but separates the code into clearly defined modules.

```
IAM Backend
 |
 +-- Authentication
 +-- Users and Profiles
 +-- Tenants and Memberships
 +-- Sessions and Devices
 +-- Authorization
 +-- Consent
 +-- Audit
 +-- Agent Delegation
```

Each module has its own responsibilities and interfaces. The modules can later be extracted into services if there is a reason to do so.

4.4 Comparison

Criterion	Microservices	Traditional monolith	Modular monolith
Development effort	High	Low	Medium
Security learning value	Medium	Medium	High
Debugging	Harder	Easy	Relatively easy
Deployment effort	High	Low	Low
Module separation	Strong	Weak unless carefully designed	Strong
Eight week risk	High	Low	Low to medium
Future extraction	Natural	Harder	Natural

4.5 Chosen Architecture

The selected approach is a modular monolith.

This is not because microservices are bad. Microservices are useful when there are independent teams, strong scaling requirements, independent deployment needs or other operational reasons. Those reasons are not important enough for this project.

The modular monolith allows the project to preserve the logical architecture while avoiding unnecessary distributed system work.

5 Authentication Architecture

5.1 Why Not Build a Full OIDC Server

The original design described an Authentication Service that would speak OIDC and issue tokens. This is possible in theory, but a secure identity provider requires many details that are easy to implement incorrectly.

OIDC is an identity layer on top of OAuth 2.0. It allows a client to verify the identity of the end user and receive identity claims from an authorization server. The OIDC Core specification defines the ID token and authentication behaviour.

OAuth 2.0 security has also evolved. RFC 9700, published in January 2025, is the current OAuth 2.0 Security Best Current Practice document. It recommends protections such as PKCE for authorization code flows, exact redirect URI matching and measures against token replay. [4]

Implementing these correctly is a separate security project.

5.2 External Identity Provider Option

The selected approach is to integrate an identity provider such as Keycloak.

The flow is:

```
Web or Mobile Client
        |
        | OIDC login
        v
External Identity Provider
        |
        | ID/access token
        v
IAM Platform
        |
        +-- User mapping
        +-- Tenant membership
        +-- Authorization
        +-- Session policy
        +-- Audit
```

The project will therefore demonstrate how an enterprise identity system can sit above an identity provider rather than pretending that the project itself is a commercial identity provider.

5.3 Responsibilities

The external identity provider handles standards heavy identity functions such as authentication and OIDC protocol behaviour.

The project handles:

- Mapping external identities to local users.
- Organization membership.
- Tenant context.
- Fine grained authorization.
- Application session and device rules.
- Consent and audit.
- Agent delegation.

6 Identity and Tenant Data Model

The main design principle is that a person, an authentication identity, an application profile and a tenant membership are different things.

6.1 User

A User represents the person in the platform.

```
User
----
id
status
created_at
updated_at
```

The User does not directly contain an organization role.

6.2 AuthenticationIdentity

An AuthenticationIdentity represents an external or local method used to identify a User.

```
AuthenticationIdentity
----------------------
id
user_id
provider
provider_subject
created_at
```

The pair of provider and provider subject should be unique.

A single user may have more than one authentication identity.

6.3 Profile

The Profile contains application information.

```
Profile
-------
user_id
display_name
email
department
```

```
avatar
updated_at
```

This separation is useful because authentication and application profile data have different responsibilities.

6.4 Organization

An Organization is a tenant.

```
Organization
------------
id
name
slug
status
domain
created_at
updated_at
```

6.5 Membership

Membership is the relationship between a User and an Organization.

```
Membership
----------
id
user_id
organization_id
status
created_at
```

The important rule is that one User can have multiple memberships.

For example:

```
User A
 |
 +-- Organization A -> Instructor
 |
 +-- Organization B -> Learner
```

This prevents roles from becoming global to a user.

6.6 Role

A Role is a tenant scoped collection of permissions.

Examples include learner, instructor and organization administrator.

The same person can therefore have different roles in different organizations.

7 Tenant Isolation

Tenant isolation is one of the most important security requirements of the system. The Week 0 report identified it as a major challenge, but Week 1 must define the actual enforcement design.

7.1 The Problem

A dangerous design would trust a tenant ID supplied by the client.

For example:

```
GET /organizations/org-B/courses
tenantId = org-B
```

A user belonging to Org A could simply change the value to Org B.

Therefore the server must not treat the client supplied tenant identifier as the source of truth.

7.2 Trusted Tenant Context

The proposed flow is:

```
Request
  |
  v
Validate authentication
  |
  v
Identify user
  |
  v
Load active membership
  |
  v
Determine organization context
  |
  v
Check resource organization
  |
  v
Authorization decision
```

The resource itself also contains its organization identifier.

For example, a course record has an organization$_id.TheAPIchecksthattheauthenticatedcontextisallowedtooper$

7.3 Tenant Isolation Alternatives

The main alternatives are shared database with tenant IDs, separate schemas for each tenant, and separate databases for each tenant.

Separate databases provide stronger physical separation, but are operationally expensive and not needed for a project of this size.

The selected approach is a shared PostgreSQL database with explicit organization IDs and centralized tenant checks. The important part is not just the database structure. The application must consistently enforce the tenant boundary.

8 Authorization Design

The project requires fine grained authorization and the Week 0 report identified three models: RBAC, ABAC and ReBAC.

These models should not be treated as three unrelated systems.

8.1 RBAC

Role Based Access Control decides access based on a role.

Example:

```
User role = Instructor
Action = create course
Decision = Allow
```

RBAC is useful for common permissions that apply to a whole role.

8.2 ABAC

Attribute Based Access Control uses attributes about the subject, resource or request context.

For example:

```
role = Instructor
organization = same
time = before grading deadline
device = trusted
```

All conditions may be required for an action.

ABAC is useful where access depends on context rather than only the user's role.

8.3 ReBAC

Relationship Based Access Control uses relationships between subjects and resources.

A good example is course grading.

```
Instructor A
     |
     | teaches
     v
Course X
     |
     | contains
     v
Submission Y
```

The question is not simply whether the user is an instructor. The question is whether the instructor teaches the course that owns the submission.

This is why ReBAC is useful for the project.

The Week 0 report uses Zanzibar as the reference for this style of authorization. The Zanzibar paper describes authorization in terms of relationships between objects and users. The project will use the idea at a much smaller scale and will not attempt to reproduce Google's system.

8.4 Unified Authorization Interface

The three models should be accessed through one interface.

```
authorize(
    subject,
    organization,
    resource,
    action,
    context
)
```

The result should contain at least:

```
decision = ALLOW or DENY
reason
```

This makes it possible to test authorization independently from the HTTP layer.

9 Session and Device Policy

The session design must be more specific than saying that tokens are short lived.

9.1 Access Tokens

Access tokens will be short lived. A proposed initial value is five minutes.

The exact value can be changed during implementation if testing shows a problem, but the final report should contain one documented value.

9.2 Refresh Tokens

Refresh tokens are associated with a session and device.

A refresh token is rotated when it is used.

```
Refresh Token A
      |
      v
Refresh
      |
      +---- Access Token B
      |
      +---- Refresh Token B
```

```
Token A is no longer valid
```

9.3 Replay Detection

If a refresh token that has already been used is presented again, this is treated as a possible replay.

The system should revoke the related refresh token family and record an audit event.

RFC 9700 specifically includes current guidance around refresh token replay and token protection. [4]

9.4 Device Revocation

Each session is linked to a device record.

A user should be able to revoke a lost device.

After revocation:

```
Device
  |
  +-- Session 1 -> revoked
  +-- Session 2 -> revoked
```

New refresh attempts from the device must fail.

The design must also define the behaviour of already issued access tokens. The chosen approach is to keep access tokens short lived and use server side session state for revocation checks where required.

9.5 Account Reset

An account reset will invalidate active sessions and refresh token families.

This gives the project a clear security property that can be tested.

10 Threat Model and Security Invariants

10.1 Assets

The important assets are:

- User identities.
- Authentication information.
- Access tokens.
- Refresh tokens.
- Tenant data.
- Membership and role information.
- Authorization policies.
- Consent records.
- Audit events.
- Agent credentials and delegations.

10.2 Threat Actors

The design considers:

- An unauthenticated attacker.
- A malicious tenant.
- A compromised user account.
- A revoked user.
- An attacker with a stolen token.
- A malicious or over privileged agent.
- A client sending malformed identity claims.

10.3 Security Invariants

The following invariants are treated as rules that must always hold.

1. A user cannot access another organization's resources without an explicitly permitted relationship.
2. A client cannot override the trusted organization context by changing a request parameter.
3. A role assigned in one organization does not automatically apply to another organization.
4. A user without an active membership cannot access tenant resources.
5. A revoked session cannot obtain a new access token through refresh.
6. Reuse of an already consumed refresh token must be detected.
7. An agent cannot have more effective permission than its delegating user.
8. Authorization fails closed when permission cannot be established.
9. Security sensitive operations generate audit events.
10. Downstream profile responses expose only fields that the caller is allowed to receive.

These invariants will later become security tests.

11 Agent Identity and Delegation

Agent identity was added after mentor feedback and is retained because it provides an interesting extension to the normal human identity model.

An agent is treated as a separate principal.

```
User
 |
 | delegates
 v
Agent
 |
 +-- scopes
 +-- expiry
 +-- revocation state
```

A delegation should contain the user, agent, organization, scopes, expiration and revocation information.

The effective permission of an agent is limited by three things:

```
Agent Permission
```

```
=
User Effective Permission
AND
Delegated Scope
AND
Tenant Context
```

For example, if a user can read a certificate but cannot delete it, an agent delegated to the user cannot gain certificate deletion permission simply because its delegation contains a delete scope.

Agent functionality will remain an extension of the core authorization system. It will not be allowed to delay the mandatory M1 to M7 requirements.

12 Enterprise Provisioning

The original Week 0 report described SCIM and a fairly complete enterprise provisioning interface. SCIM is relevant because it standardizes provisioning of users and groups between systems.

For Week 1, the project will define a minimum enterprise provisioning slice.

```
Provision Organization
        |
        v
Create organization
        |
        v
Create administrator
        |
        v
Configure domain and SSO
        |
        v
Provision employee
        |
        v
Deactivate employee
```

The provisioning operation must be idempotent.

If the same organization provisioning request is sent twice, it should not create two organizations.

This is more important for the core demonstration than implementing every SCIM feature. Full filtering, bulk operations and advanced group semantics can be added only after the main project works.

13 Consent and Audit

The Week 0 report connected the privacy part of the project to the NIST Privacy Framework. NIST describes the framework as a voluntary tool for identifying and managing privacy risk.

[7]

The project will use this as guidance rather than claiming full compliance.

Consent records should contain:

```
user
purpose
consent version
status
created_at
updated_at
```

A user should be able to change or withdraw consent.

Audit records should capture security sensitive events such as:

- Login.
- Logout.
- Token refresh.
- Token replay detection.
- Device revocation.
- Organization provisioning.
- Role changes.
- Authorization decisions where auditing is required.
- Agent creation and delegation.
- Consent changes.

The purpose is to make important events queryable rather than keeping them only in application logs.

14 Initial Authorization Scenarios

The final project requires at least 40 representative authorization scenarios. During Week 1, the first 15 scenarios are defined.

No.	Subject	Resource and action	Main model	Expected	Reason
1	Learner A	Own profile, read	RBAC/ABAC	Allow	Own profile
2	Learner A	User B profile, read	ABAC	Deny	Not permitted
3	Instructor A	Course taught by A, update	RBAC/ReBAC	Allow	Instructor teaches course
4	Instructor A	Course not taught by A, update	ReBAC	Deny	No teaching relation
5	Org admin A	User in Org A, read	RBAC	Allow	Org admin role
6	Org admin B	User in Org A, read	Tenant	Deny	Cross tenant
7	Platform admin	Organization A, read	RBAC	Allow	Platform permission

8	Learner A	Resource in Org B, read	Tenant	Deny	Cross tenant
9	Instructor A	Submission in taught course, grade	ReBAC	Allow	Valid relation
10	Instructor A	Submission in other course, grade	ReBAC	Deny	Invalid relation
11	Learner A	Own certificate, read	RBAC/ABAC	Allow	Own resource
12	Learner A	Own certificate, delete	RBAC	Deny	No delete permission
13	Agent A	Certificate, read with delegation	Delegation	Allow	Scope granted
14	Agent A	Certificate, delete without delegation	Delegation	Deny	Scope missing
15	Revoked user	Course, read	Session	Deny	Session revoked

The next weeks will expand this matrix to at least 40 cases.

15 Requirements Traceability

The project requirements will be tracked using the following structure.

Requirement	Milestone	Component	Acceptance evidence
Separate users, profiles and identities	M1	User/Profile	API and database tests
Multi organization membership	M1	Tenant	User belongs to two organizations
Consumer authentication	M2	Auth	OIDC login demonstration
Enterprise authentication	M2	Auth	Enterprise SSO simulation
Token expiry and rotation	M3	Session	Token lifecycle tests
Remote device revocation	M3	Session	Revoked device test
RBAC, ABAC and ReBAC	M4	Authorization	40+ authorization tests
Organization provisioning	M5	Tenant	Idempotency test
Consent management	M6	Consent	Consent change test
Auditability	M6	Audit	Queryable audit records

Cross tenant isolation	M7	Authorization	Attack test
Privilege escalation prevention	M7	Authorization	Negative tests
Malformed federation claims	M7	Auth	Invalid claim tests

This matrix is different from the reference mapping in Week 0. The Week 0 mapping explained which standards inform which milestones. The new matrix connects actual project requirements to implementation and tests.

16 Week 1 Implementation Skeleton

The repository will use a simple structure.

```
secure-iam-platform/
 |
 +-- src/
 |    +-- auth/
 |    +-- users/
 |    +-- tenants/
 |    +-- sessions/
 |    +-- authorization/
 |    +-- consent/
 |    +-- audit/
 |    +-- agents/
 |
 +-- tests/
 |    +-- security/
 |    +-- authorization/
 |
 +-- docs/
 |    +-- architecture.md
 |    +-- data-model.md
 |    +-- threat-model.md
 |    +-- session-policy.md
 |    +-- authorization-matrix.md
 |    +-- traceability.md
 |
 +-- prisma/
 +-- README.md
 +-- docker-compose.yml
```

The minimum vertical slice for Week 1 is:

```
Create User
   |
Create Organization
   |
```

```
Create Membership
   |
Assign Role
   |
API Request
   |
Tenant Check
   |
Authorization Check
   |
Allow or Deny
   |
Audit Event
```

This is intentionally small. It proves that the main modules can work together without requiring the full final platform.

17 Measurable Success Criteria

The project needs measurable targets instead of only using words such as secure or fine grained.

Initial targets are:

- At least 40 authorization scenarios by the final phase.
- All defined cross tenant attack tests must be rejected.
- Refresh token reuse must be rejected.
- A revoked device must not be able to refresh its session.
- Organization provisioning must be idempotent.
- Required security sensitive events must produce audit records.
- Agent permissions must never exceed the effective permission of the delegating user.
- Authorization must fail closed when a decision cannot be established.

A latency target can be measured after the first working authorization engine is available. It should not be selected before there is a basic implementation because an arbitrary number would not be very useful.

18 Questions From the Mentor and Design Answers

18.1 Can one user belong to two organizations?

Yes. A User can have multiple Membership records. Roles are evaluated in the organization context of the membership.

18.2 What prevents a tenant ID from being replaced by a client?

The server does not trust a client supplied tenant value. The organization context is derived from authenticated identity and membership, and the resource organization is checked before authorization.

18.3 What happens to an access token after device revocation?

The design uses short lived access tokens and server side session state. The session becomes invalid immediately, while an already issued access token is limited by its short lifetime and any required session check.

18.4 Which example needs ReBAC?

Instructor access to a submission based on whether the instructor teaches the course containing that submission is a clear ReBAC example.

18.5 Which OIDC responsibilities are delegated?

Protocol heavy authentication and identity provider behaviour are delegated to the selected identity provider. Application user mapping, tenant membership and project specific authorization remain inside the project.

18.6 How is organization provisioning idempotent?

A stable organization identifier or unique domain/slug is used to detect an existing organization. Repeating the same provisioning operation returns or updates the existing organization instead of creating another one.

18.7 Can an agent exceed its user's permissions?

No. The effective agent permission is limited by the user's effective permission, the delegation scope and the organization context.

18.8 What happens if authorization is unavailable?

Protected operations fail closed. The system must not assume permission simply because the authorization service or module is unavailable.

18.9 What is the smallest end to end demonstration?

The smallest demonstration is a user belonging to an organization making an API request to a tenant owned resource. The request is authenticated, the membership is checked, authorization is evaluated and the result is audited.

19 Plan for the Remaining Weeks

Week	Focus	Main deliverable
Week 1	Detailed design	Architecture, ER model, threat model, policies, traceability and initial repository
Week 2	Core identity and tenant model	PostgreSQL schema, users, profiles, organizations, memberships and roles
Week 3	Authentication and sessions	OIDC integration, token validation, sessions, devices and refresh rotation
Week 4	Authorization	RBAC, ABAC, ReBAC and initial policy engine

Week 5	Enterprise and agents	Provisioning, SSO simulation and agent delegation
Week 6	Privacy and audit	Consent, field visibility and audit queries
Week 7	Security validation	40+ scenarios and adversarial testing
Week 8	Integration and final demonstration	Complete end to end system, documentation and final demo

Security testing will not wait until Week 7. Each week will add tests for the security properties introduced in that week.

20 Conclusion

The main result of Week 1 is not a large amount of code. It is a more precise and realistic design that can guide the remaining implementation.

The Week 0 report established a broad and technically ambitious project. It correctly identified the separation between authentication, profile, membership and authorization, and it connected the project to OIDC, OAuth security, Zanzibar, SCIM and the NIST Privacy Framework.

The mentor review showed that the main risk is trying to implement too much infrastructure at the same time. The Week 1 design therefore keeps the important security and identity goals but changes the implementation strategy.

The system will initially use a modular monolith instead of eight independently deployed services. An external identity provider can handle the difficult standards based authentication functions. The project will concentrate on tenant membership, tenant isolation, fine grained authorization, session and device behaviour, enterprise provisioning, consent, audit and agent delegation.

The data model is also made more precise. User, authentication identity, profile, organization and membership are separate concepts. This allows one user to belong to several organizations with different roles. Tenant isolation is treated as an enforcement mechanism rather than only a requirement. RBAC, ABAC and ReBAC are given different purposes and are combined through a common authorization interface.

Finally, the project is being designed around tests from the start. The eventual goal is not only to show that the happy path works, but to demonstrate that cross tenant access, privilege escalation, token replay, revoked sessions and malformed claims are rejected.

This gives the project a smaller and more realistic implementation boundary while keeping the main learning goals of the original problem.

References

[1] K. Teja and J. Prakash, *A Secure Multi-Tenant Identity, Authorization and Enterprise Access Platform: Phase 1 Background, Problem Understanding and Related Work*, Indian Institute of Technology Hyderabad, September 2026.

[2] Project Mentor Review, *Review of Week 0 Project Report and Requirements for Week 1*, Indian Institute of Technology Hyderabad, September 2026.

[3] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros and C. Mortimore, *OpenID Connect Core 1.0*, OpenID Foundation, 2014. https://openid.net/specs/openid-connect-core-1_0.html

[4] T. Lodderstedt, J. Bradley, A. Labunets and D. Fett, *Best Current Practice for OAuth 2.0, RFC 9700*, IETF, January 2025. https://www.rfc-editor.org/rfc/rfc9700.html

[5] R. Pang et al., *Zanzibar: Google's Consistent, Global Authorization System*, USENIX ATC, 2019. https://www.usenix.org/conference/atc19/presentation/pang

[6] P. Hunt et al., *System for Cross-domain Identity Management: Protocol, RFC 7644*, IETF. https://www.rfc-editor.org/rfc/rfc7644.html

[7] National Institute of Standards and Technology, *NIST Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management*, NIST, 2020. https://www.nist.gov/privacy-framework